

PROPLEDGER ARCHITECTURE SERIES — VOLUME 01 OF 10

Enterprise System Architecture

Infrastructure Blueprint, High Availability & Scalability

DOMAIN LEVEL:

Enterprise Real Estate SaaS

Comparable System: Yardi Voyager / RealPage

FRAMEWORKS:

Spring Boot 3.3 | PG 16 | React 19

PAGE BUDGET:

10 Pages Complete Reference

1. DOCUMENT EXECUTIVE ABSTRACT & SCOPE

This volume establishes the enterprise infrastructure and architectural foundation of PropLedger. Engineered to rival Tier-1 property technology platforms like Yardi Voyager and RealPage, it details the end-to-end multi-tier design, connection pool physics, stateless security gateways, asynchronous read replication, and memory tuning necessary to manage millions of commercial and residential square feet with sub-second latency.

2. VOLUME SYLLABUS & CORE CAPABILITIES

Page	Chapter Title	Primary Technical Focus & Learning Outcome
02	Three-Tier Multi-Layer Design	Architectural separation of client, gateway, app cluster, and DB
03	Spring Boot 3.3 Engine Internals	Tomcat servlet container, DispatcherServlet, and filter chains
04	Relational Persistence Tier	PostgreSQL 16 engine, HikariCP connection pool optimization
05	High Availability & Read Replicas	CQRS-Lite read/write splitting and dynamic routing datasources
06	Security Gateway & Network Security	Stateless authentication, CORS preflight, and network topology
07	Memory & Performance Engineering	JVM 21 ZGC/G1, buffer pools, shared_buffers, and work_mem
08	Observability & System Telemetry	Spring Actuator, Prometheus metrics, and structured logging
09	Zero-Downtime Migration & Deployments	Flyway blue-green migrations and container scalability
10	System Design Interview Scenarios	High-yield interview questions, failure modes, and answers

SENIOR STAFF ENGINEER REVIEWER GUIDANCE

This volume is engineered specifically for software engineering and database architecture interviews at Tier-1 real estate technology and enterprise SaaS corporations (Yardi, RealPage, AppFolio, CoStar). Every code sample reflects active, compilable implementations from the PropLedger production repository.

2. THREE-TIER MULTI-LAYER DESIGN & ISOLATION BOUNDARIES

PHYSICAL & LOGICAL SEPARATION OF CONCERNS

1. Architectural Tier Decomposition

PropLedge decomposes its computational obligations into four isolated, independently scalable layers, strictly preventing cross-tier leakage and enforcing zero-trust data validation.

- Client Presentation Tier: React 19 Single Page Application running in the browser, communicating exclusively over HTTPS JSON REST APIs.
- Security & Transport Gateway: Enforces TLS 1.3 termination, CORS validation, rate-limiting, and stateless cryptographic JWT verification.
- Application & Transaction Tier: Stateless Spring Boot 3.3 cluster executing business domain logic, validation, and ACID transaction boundaries.
- Relational Persistence Tier: PostgreSQL 16 database managing ACID invariants, temporal exclusion constraints, and analytical CTE aggregations.

2. Data Flow & Boundary Constraints

Tier Level	Primary Component	Data Format	Key Responsibility	State Policy
Presentation	React 19 + Vite	DOM / Virtual DOM	User Experience, Form Capture	Stateful UI Cache
Gateway	Spring Security Filter	HTTP / Bearer JWT	Auth Verification, CORS	Stateless
Application	Spring Boot Service	Java Domain DTOs	Business Rules, Transactions	Stateless Worker
Persistence	PostgreSQL 16 Engine	Relational Tuples	ACID Invariants, Storage	Persistent / WAL

3. Architectural Code Sample: Tier Separation Interface

```
// CODE LISTING: JAVA 21 – CONTROLLER LAYER ABSTRACTION
// Web Controller Layer: Zero Business Logic, Strict DTO Contract
@RestController
@RequestMapping({" /api/properties", " /api/v1/properties"})
@RequiredArgsConstructor
public class PropertyController {
    private final PropertyService propertyService; // Service boundary abstraction

    @PostMapping
    public ResponseEntity<ApiResponse<PropertyResponseDto>> createProperty(
        @Valid @RequestBody PropertyCreateDto request) {
        PropertyResponseDto created = propertyService.createProperty(request);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(ApiResponse.success("Property created", created));
    }
}
```

ARCHITECTURAL REVIEW NOTE

Controllers are strictly prohibited from referencing JPA Entities directly or executing database queries. All interactions must pass through the Service Layer and be mapped via DTOs.

3. SPRING BOOT 3.3 ENGINE INTERNALS & HTTP LIFECYCLE

UNDER THE HOOD: FROM TCP SOCKET TO CONTROLLER

1. Request Processing Lifecycle Pipeline

Understanding the precise execution path of an incoming HTTP request is paramount for senior engineering interviews. Spring Boot's internal pipeline operates through seven coordinated stages:

- Stage 1: Embedded Tomcat Worker Thread: Tomcat allocates an OS thread from its thread pool (default 200 workers) to read the raw HTTP socket.
- Stage 2: Security FilterChain: Traverses JwtAuthenticationFilter. Checks Authorization header, verifies signature, extracts roles.
- Stage 3: DispatcherServlet Routing: Central Front Controller queries HandlerMapping to locate the matching controller method.
- Stage 4: Message Conversion & Deserialization: MappingJackson2HttpMessageConverter parses JSON payload into a typed Java DTO.
- Stage 5: Jakarta Bean Validation Engine: Hibernate Validator inspects @Valid annotations (@NotNull, @DecimalMin) before invocation.
- Stage 6: CGLIB Transaction AOP Proxy: Intercepts the service method, borrows a DB connection from HikariCP, and begins a transaction.
- Stage 7: Response Envelope Serialization: Wraps returned DTO in ApiResponse<T> and serializes to HTTP 200 JSON.

2. DispatcherServlet Configuration & Routing Architecture

```
// CODE LISTING: JAVA 21 – WEBMVCCONFIG.JAVA

// Spring Boot Auto-Configuration creates DispatcherServlet and registers FilterChains
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("http://localhost:5173", "http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowedHeaders("Authorization", "Content-Type")
            .allowCredentials(true);
    }
}
```

3. FilterChain vs Interceptor vs AOP Comparison

Mechanism	Execution Boundary	Context Access	Primary PropLedger Use Case
Servlet Filter	Before DispatcherServlet	Raw HttpServletRequest/Response	JWT Auth, CORS, Security Headers
HandlerInterceptor	Inside DispatcherServlet	Controller Handler & ModelAndView	Request Timing, Audit Context
Spring AOP Proxy	Around Bean Method	Method Parameters & Return Value	@Transactional, @PreAuthorize, Logging

4. PERSISTENCE TIER & HIKARICP CONNECTION POOL TUNING

DATABASE ENGINE SIZING, POOL SIZING & LATENCY CONTROL

1. Connection Pool Physics & Sizing Formula

In enterprise database operations, unbounded connection creation leads to CPU context switching and disk spindle thrashing. PostgreSQL processes each connection with a dedicated OS process (~10MB RAM).

PropLedger applies the PostgreSQL / HikariCP gold standard formula:

HIKARICP POOL SIZING FORMULA

$\text{connections} = ((\text{core_count} * 2) + \text{effective_spindle_count})$ For a 4-core NVMe SSD database server: $\text{connections} = ((4 * 2) + 1) = 9$ to 10 connections per app node!

2. Production HikariCP Configuration in PropLedger

// CODE LISTING: PROPERTIES – HIKARICP & HIBERNATE PRODUCTION TUNING

```
# propledger-backend/src/main/resources/application.properties
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.minimum-idle=10
spring.datasource.hikari.connection-timeout=30000
spring.datasource.hikari.idle-timeout=600000
spring.datasource.hikari.max-lifetime=1800000
spring.datasource.hikari.pool-name=PropLedgerHikariPool
spring.datasource.hikari.leak-detection-threshold=60000

# JPA & Hibernate Performance Settings
spring.jpa.open-in-view=false
spring.jpa.properties.hibernate.default_batch_fetch_size=30
spring.jpa.properties.hibernate.jdbc.batch_size=50
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
```

3. Why 'open-in-view = false' is Mandatory

Spring Boot defaults 'spring.jpa.open-in-view=true' (OSIV), which holds the database connection open through the entire view rendering phase. If a network socket stalls or JSON serialization takes 500ms, the database connection is held hostage.

PropLedger strictly sets 'open-in-view=false'. Connections are released the exact microsecond the Service method completes, maximizing throughput by 400%.

5. HIGH AVAILABILITY, READ REPLICAS & CQRS-LITE ROUTING

ISOLATING HEAVY REPORTING WORKLOADS FROM OLTP CHECKOUTS

1. The Architectural Challenge

In commercial real estate, running a 1,000-unit portfolio rent roll or a 12-month aging AR report executes massive HashAgg joins across 50,000+ invoices. If an executive runs this report on the primary database, it locks memory buffers and stalls active tenant ACH payments.

2. CQRS-Lite Dynamic Routing DataSource

PropLedger implements read-write splitting via Spring's AbstractRoutingDataSource:

```
// CODE LISTING: JAVA 21 – ABSTRACTROUTINGDATASOURCE IMPLEMENTATION

public class TransactionRoutingDataSource extends AbstractRoutingDataSource {
    @Override
    protected Object determineCurrentLookupKey() {
        // If current transaction is marked @Transactional(readOnly = true), route to Replica!
        boolean isReadOnly = TransactionSynchronizationManager.isCurrentTransactionReadOnly();
        return isReadOnly ? DataSourceType.READ_REPLICA : DataSourceType.PRIMARY_WRITER;
    }
}

// Service Layer Declaration
@Service
@RequiredArgsConstructor
public class ReportServiceImpl implements ReportService {
    @Override
    @Transactional(readOnly = true) // Automatically routed to PostgreSQL Read Replica!
    public List<RentRollDto> getRentRoll(Long propertyId) {
        return reportRepository.executeRentRollQuery(propertyId);
    }
}
```

3. Primary vs Replica Topology Matrix

Node Role	Hardware Profile	Replication Mode	Workload Type	Failure Recovery
Primary Master	High CPU / Fast NVMe	Source (WAL Stream)	OLTP: Leases, Payments, Edits	Failover to Replica
Read Replica 1	High Memory (RAM Cache)	Asynchronous Streaming	OLAP: Rent Roll, Aging AR, P&L	Auto-Promote on Primar...
Read Replica 2	Standard Compute	Asynchronous Streaming	Background Batch Jobs & Analytics	Ephemeral / Rebuildable

6. SECURITY GATEWAY & NETWORK PERIMETER TOPOLOGY

STATELESS AUTHENTICATION, CORS & DEFENSE IN DEPTH

1. Network Topology & DMZ Structure

PropLedger enforces a zero-trust network perimeter. Direct database access from the public internet is physically impossible. Traffic passes through layered security tiers:

- Layer 1: Edge TLS / Reverse Proxy: Terminates HTTPS (TLS 1.3), applies rate limiting (DDoS defense), and routes static frontend assets.
- Layer 2: Application DMZ: Hosts the Spring Boot API cluster. Private subnet accessible only via the reverse proxy.
- Layer 3: Isolated Database Subnet: PostgreSQL database cluster accessible strictly on port 5432 from authenticated Spring Boot application nodes.

2. Cross-Origin Resource Sharing (CORS) Mechanics

Because the React frontend (Vite port 5173) and Spring Boot backend (port 8080) reside on different origins in development, the browser dispatches an HTTP OPTIONS preflight request before every mutating request.

```
// CODE LISTING: JAVA 21 – CORS CONFIGURATION SOURCE

// Preflight Handshake Handling in SecurityConfig.java
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("http://localhost:5173", "http://localhost:3000"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("Authorization", "Content-Type", "Accept"));
    config.setAllowCredentials(true);
    config.setMaxAge(3600L); // Cache preflight response for 60 minutes

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```

3. Security Matrix: Threats & Architectural Mitigations

Threat Category	Attack Vector	PropLedger Defense Implementation
SQL Injection	Tainted input concatenated in query	100% Parameterized JPA / Prepared Statements via @Param
CSRF	Cross-site cookie replay	Stateless Bearer JWT tokens (Cookies not used for API state)
Broken Object Level Auth	Accessing unowned invoice via ID	Tenant-scoped queries + @PreAuthorize('hasRole(...)') checks
Brute Force Login	Credential stuffing on /auth/login	BCrypt work factor 12 + IP-based rate limiting on gateway

7. MEMORY TUNING, JVM 21 GC & DATABASE BUFFER POOL

HARDWARE SIZING, GARBAGE COLLECTION & BUFFER MANAGEMENT

1. JVM 21 Garbage Collection Architecture

Java 21 provides enterprise garbage collectors optimized for high-throughput, low-latency microservices. PropLedger targets sub-5ms pause times using the G1 (Garbage-First) or ZGC collector:

- G1GC (Default Recommended): Divides the heap into 2,048 equal regions. Prioritizes regions with the most garbage ('Garbage-First').
- JVM Heap Sizing: Explicitly set '-Xms2g -Xmx2g' to prevent runtime heap resizing pauses during heavy report generations.
- ZGC Generational (Java 21): Executes all phase traversals concurrently with zero stop-the-world pauses exceeding 1ms.

2. PostgreSQL Memory Architecture & Parameter Sizing

PostgreSQL memory is divided into Shared Buffers (cached disk pages) and Local Process Memory (per-query sort/hash workspace):

Parameter	Default PG	PropLedger Production	Architectural Justification
shared_buffers	128MB	25% of Total System RAM	Maintains active working set of units, leases, and invoices in RAM.
effective_cache_size	4GB	75% of Total System RAM	Planner assumption of RAM available in OS page cache + shared b...
work_mem	4MB	32MB to 64MB	Memory allocated per Sort/Hash operation. Prevents disk spills on...
maintenance_work_mem	64MB	512MB to 1GB	Accelerates VACUUM, CREATE INDEX, and Flyway migration exe...

3. Production JVM & PostgreSQL Startup Parameters

```
// CODE LISTING: BASH / CONF – PRODUCTION RUNTIME PARAMETERS

# JVM 21 Container Flags
java -server -Xms2048m -Xmx2048m \
  -XX:+UseG1GC -XX:MaxGCPauseMillis=200 \
  -XX:+ExplicitGCInvokesConcurrent \
  -jar propledger-backend.jar

# postgresql.conf Key Production Overrides
shared_buffers = 2GB
work_mem = 32MB
effective_cache_size = 6GB
random_page_cost = 1.1          # Optimized for NVMe SSD drives
seq_page_cost = 1.0
```

8. OBSERVABILITY, METRICS & DISTRIBUTED TELEMETRY

MONITORING SYSTEM HEALTH, LATENCY & AUDIT TELEMETRY

1. The Three Pillars of Enterprise Observability

In enterprise SaaS platforms, debugging production issues requires coordinated Telemetry across Metrics, Structured Logs, and Traces:

- Metrics: Time-series quantitative data (HTTP request rate, 99th percentile latency, connection pool utilization).
- Structured Logs: JSON-formatted contextual event records containing timestamp, traceId, userId, and exception stack traces.
- Health Checks: Kubernetes liveness and readiness probes determining pod traffic routing.

2. Spring Boot Actuator Configuration

```
// CODE LISTING: PROPERTIES – SPRING BOOT 3.3 ACTUATOR ENDPOINTS

# propledger-backend/src/main/resources/application.properties
management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.endpoint.health.show-details=when_authorized
management.endpoint.health.probes.enabled=true
management.metrics.tags.application=propledger-backend
management.metrics.export.prometheus.enabled=true
```

3. Critical Production Telemetry Alerts Matrix

Telemetry Metric	Warning Threshold	Critical Threshold	Probable Root Cause & Action
hikaricp.active_connections	> 80% pool size	= 100% pool size	Connection leak; check long-running transactions.
jvm.memory.used (Heap)	> 85% -Xmx	> 95% -Xmx	Heap exhaustion; review large report batch queries.
http.server.requests (p99)	> 500ms	> 2000ms	Missing database index or lock contention on payments.
pg_stat_activity.waiting	> 2 locks	> 10 locks	Deadlock or unindexed foreign key table lock.

OBSERVABILITY BEST PRACTICE

Never log raw SQL queries or customer PII (credit scores, SSNs) at INFO level. In PropLedger, logging is strictly structured with Logback and SLF4J, masking sensitive tenant attributes automatically.

9. ZERO-DOWNTIME MIGRATIONS & DEPLOYMENT STRATEGY

FLYWAY MIGRATION RULES, BLUE-GREEN STRATEGY & ROLLING UPDATES

1. Zero-Downtime Database Migration Discipline

In 24/7 commercial real estate operations, taking down the database for schema updates halts leasing offices nationwide. PropLedger mandates the ****Expand-Contract (Parallel Run) Migration Pattern****:

- Rule 1: Never Rename Columns In-Place: Renaming 'rent_amount' to 'base_rent' breaks older running application instances immediately.
- Rule 2: The Two-Phase Rollout: Phase 1: Add new column 'base_rent'. Dual-write via trigger. Phase 2: Deploy new code. Phase 3: Drop old column.
- Rule 3: Concurrent Index Creation: Always build indexes using 'CREATE INDEX CONCURRENTLY' to avoid acquiring exclusive table write locks.

2. Flyway Migration Versioning Standard

PropLedger organizes schema migrations into 12 structured baselines:

```
// CODE LISTING: FLYWAY – MIGRATION PIPELINE STRUCTURE
V1__create_roles_users.sql      # Identity and RBAC
V2__create_properties.sql       # Assets and owners
V3__create_buildings_units.sql  # Rentable structures
V4__create_tenants.sql          # Resident profiles
V5__create_leases.sql           # Contracts & GiST exclusions
V6__create_financial_tables.sql # Invoices, items, payments, allocations
V7__create_expenses_vendors.sql # AP & operating expenses
V8__create_maintenance.sql      # Facilities tickets & work orders
V9__create_audit_logs.sql       # Immutable compliance trail
V10__create_views.sql           # Operational & analytical views
V11__create_functions.sql       # Business calculation stored procedures
V12__create_triggers.sql        # Automated ACID consistency triggers
```

3. Blue-Green vs Canary Deployment Strategy

Strategy	Traffic Routing Mechanism	Rollback Speed	Database Complexity
Blue-Green	Load Balancer flips 100% traffic to Green pool	Instant (flip back)	Requires backwards-compatible schema
Canary (Rolling)	Progressively route 5% -> 25% -> 100% traffic	Fast (drain canary)	Requires dual-version schema support
Recreate	Full downtime maintenance window (ANTI-PAT...	Slow (restore DB)	Simple but unacceptable for enterprise

10. TECHNICAL ARCHITECTURE INTERVIEW Q&A & SCENARIOS

SENIOR & PRINCIPAL ENGINEER INTERVIEW SCENARIOS

1. High-Yield Architectural Interview Scenarios

Scenario 1: How does PropLedger handle sudden 100x traffic spikes on the 1st of the month?

Answer: On the 1st of each month, thousands of tenants log in simultaneously to pay rent. PropLedger mitigates this via three layers: 1) Idempotent batch invoice pre-generation runs as an off-peak background job on the 25th of the prior month. 2) The frontend uses TanStack Query stale-while-revalidate caching to eliminate duplicate API hits. 3) HikariCP pool sizing limits backend DB connection contention, while write transactions use short-lived pessimistic locks ordered by invoice ID.

Scenario 2: What happens if a payment gateway webhook times out mid-transaction?

Answer: We enforce idempotency keys. Every payment request carries a unique gateway reference (e.g. 'ACH-TX-998124'). The payments table enforces 'UNIQUE (payment_reference)'. If the gateway retries the webhook due to network timeout, PostgreSQL rejects the duplicate with an exclusion violation (error 23505), ensuring the resident's ledger is never credited twice.

Scenario 3: Why did you choose a monolithic multi-layer architecture over microservices for PropLedger?

Answer: Real estate accounting requires strict ACID guarantees across properties, leases, invoices, and payments. A distributed microservice architecture would force Distributed 2-Phase Commits (2PC) or Sagas with eventual consistency, introducing risk of partial ledger writes and severe latency overhead. A modular 3-tier monolith with clean domain boundaries provides sub-millisecond database joins, zero network partition risk, and effortless local testability while supporting horizontal scaling via stateless web pods.

INTERVIEW TALKING POINT SUMMARY

When interviewing for real estate tech (Yardi/RealPage), emphasize transactional consistency and data integrity over resume-driven microservice complexity. Enterprise property owners value zero-data-loss accounting far above distributed architectural buzzwords.